Demo app: pharmacy-api (Spring Boot)

Module 5: Continuous Delivery, Environments & Deployment Strategies

Day 3 · Azure DevOps, CI/CD and AI-Assisted Release Readiness

What We'll Cover

1. Multi-stage YAML for Continuous Delivery
2. Environments, approvals and checks
3. Deployment strategies — rolling, blue-green and canary
4. Release gates and automated pre-/post-deployment checks
5. Secrets and configuration per environment
6. Rollback and progressive delivery
7. Human approval gates for production promotion
8. Case study: Dev auto-deploy, a canary to staging, and a controlled rollback

Where Module 4 Left Off

- Module 4's gate proved the code is good enough to ship — tests pass, coverage clears, SonarCloud is green
- “Good enough to ship” and “actually shipped safely” are two different problems
- Module 5 is the second one: get it onto real environments, in front of real traffic, without a bad deploy taking the system down

Today's question

If a build passes every gate, should it go straight to production? (No — it goes to Dev, then Staging with a human watching, then Prod with a human deciding.)

Multi-Stage YAML for Continuous Delivery

- Same azure-pipelines.yml, more stages — CI stages (Build/Test/Quality Gate) feed into CD stages (DeployDev, DeployStaging, DeployProd)
- Each CD stage depends on the one before it: dependsOn + a condition (usually succeeded())
- A deployment job (deployment:, not job:) is what lets a stage target an environment — that's what unlocks approvals, checks, and deployment history
- The artifact that moves between stages is the same drop produced back in Module 3 — CD deploys what CI already proved, it doesn't rebuild it

Shape of it

```
stages:
- Build
- Test
- QualityGate
- DeployDev
- DeployStaging
- DeployProd
```

Each deploy stage: one deployment job, targeting one environment.

Environments, Approvals and Checks

- An environment (Pipelines → Environments) is its own Azure DevOps object — Dev, Staging, Prod — separate from the pipeline YAML
- Approvals and checks are configured on the environment, not the stage — so the same gate applies no matter which pipeline deploys to it
- Approval: a named person/group must sign off before the deployment job runs
- Checks: business-hours windows, a required template, invoking another pipeline, or an automated REST API call that must return success

Why this split matters

Put the approval on the environment once, and every pipeline that ever deploys to Staging or Prod inherits it — nobody can quietly add a pipeline that skips the gate.

Deployment Strategies

Rolling

- Replace instances a few at a time
- Simple, no extra infrastructure
- A bad version still reaches some real users before anyone notices

Blue-Green

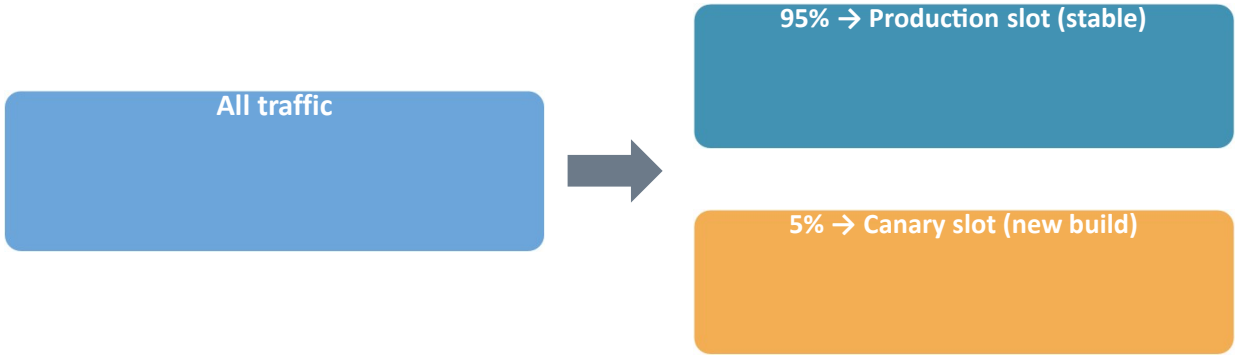
- Two full environments; switch traffic all at once
- Instant rollback — just switch back
- Costs double the infrastructure while both exist

Canary

- Route a small % of traffic to the new version first
- Bad version only affects that small %, caught early
- Needs real traffic-splitting (slots, service mesh, feature flags)

This case study uses canary — the deck's going to spend the next slide on it specifically.

Canary, in Detail



- Deploy the new build to a separate slot — production traffic isn't touching it yet
- Route a small percentage of real traffic to that slot
- Watch an error budget over a set window of requests — this case study landed on 20%, not the 2% a textbook default would suggest (next slide covers why)
- Pass → shift more traffic, eventually swap the canary into production; fail → route 100% back to the stable slot

In Azure Pipelines

AzureCLI@2 steps: az webapp traffic-routing set to split %, then az webapp deployment slot swap to promote once it's proven.

Lessons From the Live Canary Run

Cold-Start 504

The very first canary request hit a JVM boot + App Service cold start — it timed out before the app was even up, and got counted as a failure.

Fix: a warm-up loop polls the slot for a first 200 before the error-budget window starts.

Transient Blips

A stray curl connection reset or DNS hiccup during the measurement window — infrastructure noise, unrelated to app health.

Fix: one immediate retry before counting a failure. A genuinely broken app still fails both tries.

2% → 20% Threshold

The textbook 2% budget tripped repeatedly on this shared training environment's own jitter, not a real app defect.

Fix: recalibrated to the honest noise floor for this environment — found by watching real runs, not guessed.

None of this loosens the gate — the threshold is still enforced in code and still fails the stage when a build is genuinely broken (Lab 4, next).

Release Gates & Deployment Checks

- A pre-deployment check runs before the deployment job — e.g. confirm the target environment is healthy before touching it
- A post-deployment check (a smoke test) runs right after — hit a real endpoint on the freshly deployed instance and confirm it actually responds
- pharmacy-api already exposes /actuator/health for exactly this — the same endpoint Module 4's integration test checked
- A smoke test that only checks “did the deploy task report success” isn't enough — it proves the deploy command ran, not that the app works

Case study tie-in

The Dev stage's auto-deploy ends with a script step calling GET /actuator/health on the deployed app and failing the stage if it isn't UP.

Secrets & Configuration per Environment

- Dev, Staging and Prod need different values for the same setting — a connection string, an API base URL, a feature flag
- Variable groups (Pipelines → Library) scoped per environment — dev-config, staging-config, prod-config — keep them separate and auditable
- A secret variable (or one backed by Azure Key Vault) is never printed in a log and must be explicitly mapped into a step's env:
- Never bake an environment's config into the built artifact — the same drop has to be deployable to all three

Common mistake

Hard-coding a Dev URL into application code “just for now.” It always outlives “for now,” and it's exactly how a Dev secret ends up reachable from Prod.

Rollback & Progressive Delivery

- Rollback = getting back to the last known-good state fast, without a fire-drill
- Blue-green: rollback is just re-pointing traffic at the environment that was never touched
- Canary: rollback is routing 100% of traffic back to the stable slot — the bad build never fully replaced it
- Progressive delivery is the general idea behind both: expose a change to a little bit of reality at a time, and make undoing it cheap

Case study tie-in

Lab 4 (Module 5) — next up — breaks the canary slot on purpose, watches the error budget trip, then triggers the rollback step by hand and confirms traffic is fully back on stable.

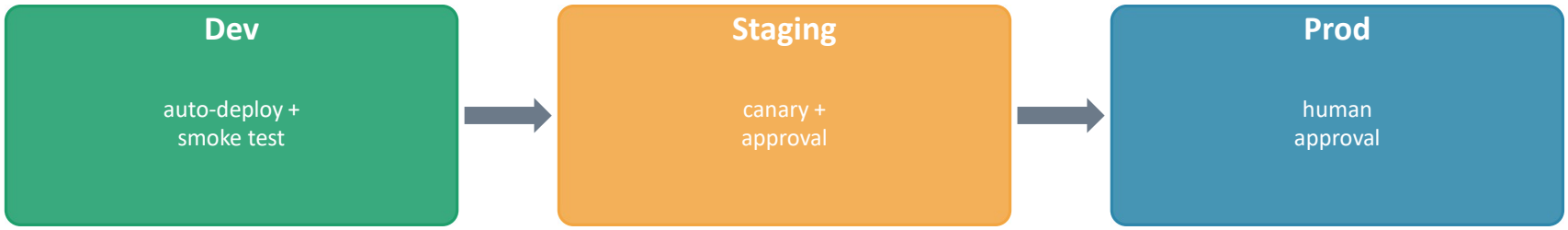
Human Approval Gates for Production

- An approval is a deliberate point where a person, not a script, decides — usually before Prod, sometimes before Staging too
- Configured on the environment (see slide 5) — one or more named approvers, optional instructions text, an optional timeout
- The approver should see the evidence, not just a button — link the SonarCloud result, the canary's error-budget numbers, the smoke test result right there
- An approval is not a rubber stamp — it's the one point a human can say no with full context, right before the change reaches real users

Teaching point

Ask learners: why put the approval on the environment instead of a manual step in every pipeline? (So nobody can add a new pipeline that quietly skips it.)

Putting It Together: The Module 5 Pipeline



Feeds in from Module 4's drop — the same artifact moves through all three environments, never rebuilt in between.

Today's Case Study

Extend the pipeline into multi-stage CD

- Deploy automatically to a Dev environment, with a smoke test gate
- Add a manual approval gate before Staging
- Run a canary release into Staging — a traffic split with a watched error budget
- Demonstrate a controlled rollback when the canary is deliberately broken — the exercise still to run

Full step-by-step is in the separate Module 5 Labs doc — this slide is the "why," not the "how."

Knowledge Check

Staging has a required approval AND a canary release step configured. A deployment job targeting Staging starts running. In what order do things happen?

A. The canary release runs first, then the approval is requested before traffic fully shifts

B. The approval is requested first; nothing in the deployment job runs until it's granted

C. Both happen at the same time, independently

D. The approval only applies to Prod, never to Staging

Answer

B — Correct

Checks and approvals on an environment gate entry into that environment — they run before any step in the deployment job executes, canary release included. Nothing in the job body runs until the approval is granted.

- A is wrong — that would let unapproved code already be serving some live traffic
- C is wrong — approvals specifically block the job from starting at all
- D is wrong — approvals can be configured on any environment, Staging included; this case study's approval sits in front of Staging on purpose

Key Takeaways

- CD is CI's artifact plus more stages — the drop that Module 3/4 proved is what moves through Dev, Staging and Prod, never rebuilt in between
- Approvals and checks belong on the environment, not the pipeline — that's what makes the gate impossible to quietly bypass
- Canary limits blast radius by limiting traffic, not by limiting trust — watch a real error budget, don't just assume it's fine
- A smoke test has to call the real app, not just check that the deploy task reported success
- Cheap rollback is a design decision made before the incident, not a plan improvised during one

End of Day 3

Azure DevOps, CI/CD and AI-Assisted Release Readiness